

PROJECT PRESENTATION

Detecting and Responding to SSH Brute-Force Attacks

using the Wazuh SIEM in a Kali Linux + Ubuntu Server Lab

Presented by

Sameeha Shajahan

Student at Brototype

Lab component	Details
Attacker machine	Kali Linux 2026.1 (VirtualBox) - 192.168.56.102
Target server	Ubuntu Server "secops-server" (VirtualBox) - 192.168.56.101
SIEM server	Wazuh v4.14.8 OVA (VirtualBox) - 192.168.56.104
Focus	Detect > Observe > Block > Document

All attacks were performed only on my own isolated virtual lab, with permission and for learning.

Table of Contents

Flowchart: How the SSH Brute-Force Attack Is Detected	3
1. Introduction	4
What is this project?.....	4
Purpose.....	4
Problem it solves	4
2. Project Overview.....	5
Scope	5
Objectives	5
Lab architecture.....	5
3. Development Process.....	6
Step 1: Network setup and connectivity test	6
Step 2: SSH service verification	7
Step 3: Attack simulation (failed login attempts).....	9
Step 4: Log collection on the Ubuntu server	9
Step 5: Verifying the logs reached Wazuh.....	10
Step 6: Wazuh dashboard analysis	12
4. Key Features	14
5. Incident Response Phase: Observe, Block and Document	15
5.1 Observing the events.....	15
5.2 Blocking criteria (when do we block an IP?).....	16
5.3 Documentation criteria (what must be written after the incident?)	17
Severity classification	18
Incident report (fill this after your final test)	18
6. Technologies Used	21
7. Challenges Faced	22
8. Achievements	22
9. Future Plans	22
10. Personal Contribution	22
11. Lessons Learned	23
12. Conclusion	23
Appendix A: Abbreviations	24

Flowchart: How the SSH Brute-Force Attack Is Detected

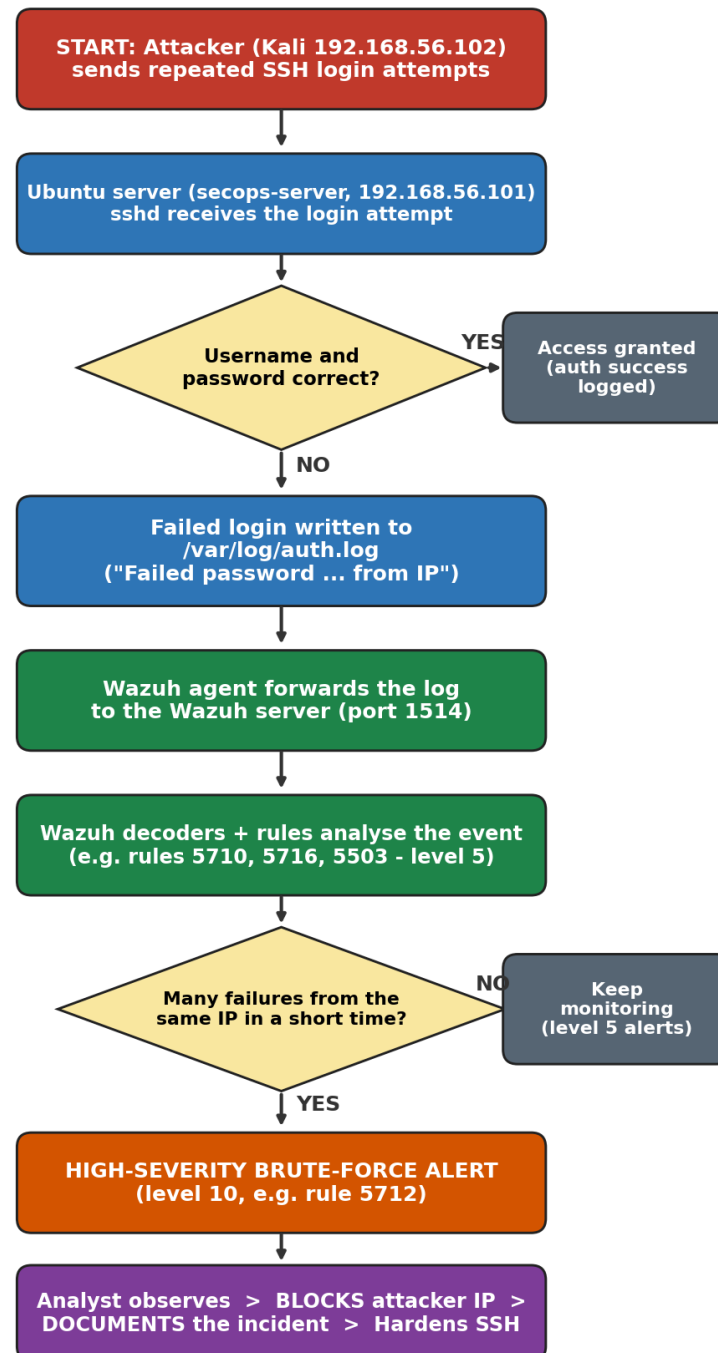


Figure 1: SSH brute-force attack and Wazuh detection/response flow

1. Introduction

What is this project?

This project builds a small security operations lab and uses it to simulate, detect, block and document an SSH brute-force attack. A brute-force attack means an attacker keeps guessing usernames and passwords until one works. The attacker (Kali Linux) targets an Ubuntu server, while Wazuh, an open-source SIEM (Security Information and Event Management) platform, watches the server logs and raises alerts.

Purpose

- Understand how a brute-force attack looks from the attacker side and, more importantly, from the defender side.
- Learn how log collection, correlation and alerting work in a real SIEM.
- Practise the full incident-response cycle: detect, observe, contain (block the IP) and document.

Problem it solves

Any server with SSH open to a network is constantly probed by password-guessing attempts. Without central monitoring, an admin may never notice thousands of failed logins hidden in `/var/log/auth.log`, and the one successful guess is the one that leads to a breach. Wazuh turns those raw log lines into clear alerts so the attack can be stopped early.

2. Project Overview

Scope

In scope	Out of scope
SSH password guessing against one Ubuntu server	Attacks on real / internet-facing systems
Log collection (auth.log) and forwarding to Wazuh	Malware, ransomware or web-application attacks
Alert analysis in the Wazuh dashboard and alert files	Large-scale distributed (botnet) attacks
Blocking the attacker IP and writing an incident report	Enterprise-scale deployment and tuning

Objectives

1. Build a three-machine virtual lab and confirm every machine can reach the others.
2. Verify that the SSH service is running and check its authentication settings.
3. Simulate failed SSH logins from Kali against the Ubuntu server.
4. Collect the resulting log entries on the Ubuntu server and in Wazuh.
5. Observe the alerts, MITRE ATT&CK mapping and event details in the Wazuh dashboard.
6. Block the attacking IP address using a clear blocking criteria.
7. Document the incident using a clear documentation criteria.

Lab architecture

Machine	Role	OS / Software	IP address
Kali VM	Attacker	Kali Linux 2026.1, OpenSSH 10.3p1	192.168.56.102
secops-server	Target + Wazuh agent	Ubuntu Server, OpenSSH server (user: samee)	192.168.56.101
wazuh-server	SIEM manager, indexer, dashboard	Wazuh v4.14.8 OVA (also NAT adapter 10.0.2.15)	192.168.56.104

Network type: all three VMs sit on a VirtualBox host-only network (192.168.56.0/24), so they can talk to each other and to the host laptop, but the attack never leaves the laptop.

3. Development Process

Methodology: I followed an iterative (Agile-style) lab approach: plan a small step, build it, test it, observe the result and fix problems before moving on. Each step below was verified with a screenshot before starting the next, so any fault was found early instead of at the end (unlike Waterfall, where testing happens last).

Tools used: VirtualBox (virtualisation), Linux terminal commands (ping, ssh, sshd -T, tail, grep, ip a), Wazuh dashboard (browser), Snapshots (to roll back the VMs).

Step 1: Network setup and connectivity test

Before any attack, every VM must reach every other VM. I used ping (ICMP echo) between Ubuntu, Wazuh and Kali in both directions. Replies with 0% packet loss prove the network and IP addressing are correct. A working network is required because the Wazuh agent sends logs over the network and Kali must reach the SSH port.

```
samee@secops-server:~$ ping -c 4 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
 64 bytes from 192.168.56.104: icmp_seq=1 ttl=127 time=4.86 ms
 64 bytes from 192.168.56.104: icmp_seq=2 ttl=127 time=1.31 ms
 64 bytes from 192.168.56.104: icmp_seq=3 ttl=127 time=1.64 ms
 64 bytes from 192.168.56.104: icmp_seq=4 ttl=127 time=1.12 ms

--- 192.168.56.104 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 2997ms
 rtt min/avg/max/mdev = 1.115/2.230/4.856/1.527 ms
samee@secops-server:~$
```

Figure 2: Ubuntu to Wazuh: ping -c 4 192.168.56.104

```
wazuh-user@wazuh-server ~1$ ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
 64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=2.49 ms
 64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=1.12 ms
 64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=2.83 ms
 64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=1.49 ms

--- 192.168.56.101 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3016ms
 rtt min/avg/max/mdev = 1.122/1.982/2.829/0.699 ms
wazuh-user@wazuh-server ~1$
```

Figure 3: Wazuh to Ubuntu: ping -c 4 192.168.56.101

```

samee@secops-server:~$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=5.67 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=2.57 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=1.93 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=2.80 ms

--- 192.168.56.102 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2997ms
rtt min/avg/max/mdev = 1.930/3.244/5.672/1.437 ms
samee@secops-server:~$

```

Figure 4: Ubuntu to Kali: ping -c 4 192.168.56.102

```

kali@kali:~$ ping -c 4 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=127 time=4.18 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=127 time=3.70 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=127 time=1.68 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=127 time=1.93 ms

--- 192.168.56.104 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.681/2.871/4.179/1.084 ms

kali@kali:~$ ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=12.3 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=3.39 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=6.09 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=15.6 ms

--- 192.168.56.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 3.387/9.342/15.619/4.850 ms

kali@kali:~$

```

Figure 5: Kali to Wazuh (.104) and Kali to Ubuntu (.101): 4 packets sent, 4 received, 0% loss

HOW TO READ THIS SCREENSHOT

4 transmitted, 4 received, 0% packet loss = the target is reachable.

time=1.6 to 15 ms = very low delay, normal for machines on the same host.

ttl=64 on 192.168.56.101 = TTL is the "Time To Live" counter of a packet; 64 is the usual starting value on Linux systems, so it fits an Ubuntu machine.

Step 2: SSH service verification

SSH (Secure Shell) is the encrypted remote-login service on port 22. I checked that the SSH client on Kali works (ssh -V shows OpenSSH_10.3p1) and that the SSH server on Ubuntu is configured. Using `sudo sshd -T` I printed the effective configuration and filtered the authentication settings.

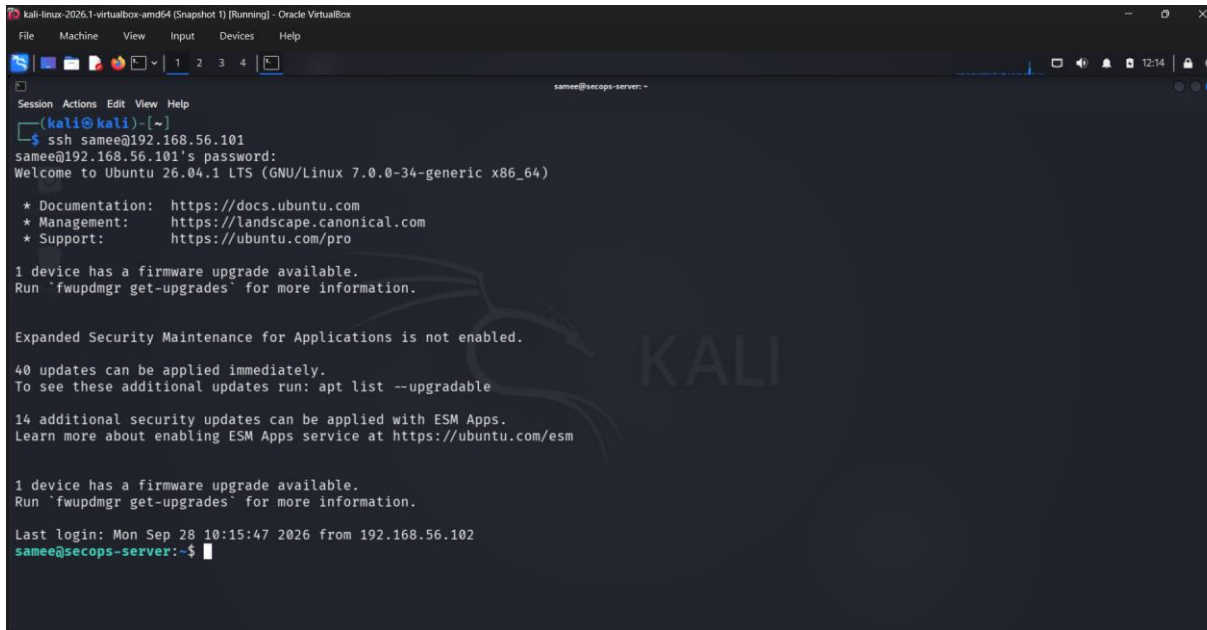


Figure 6: SSH connection test: Kali connects to Ubuntu with the correct password

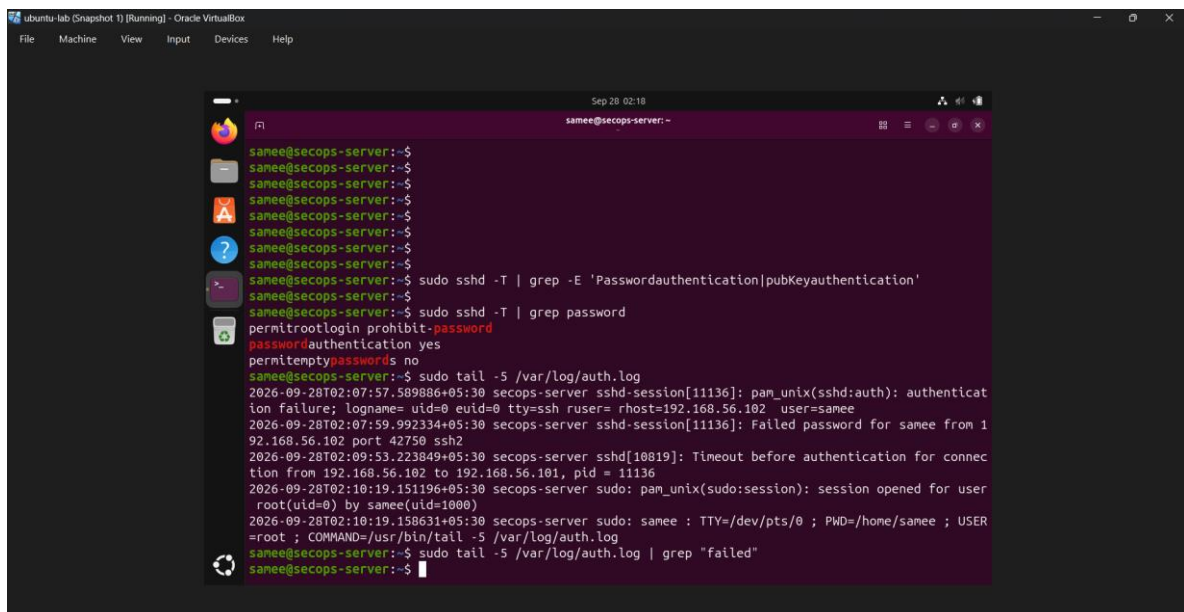
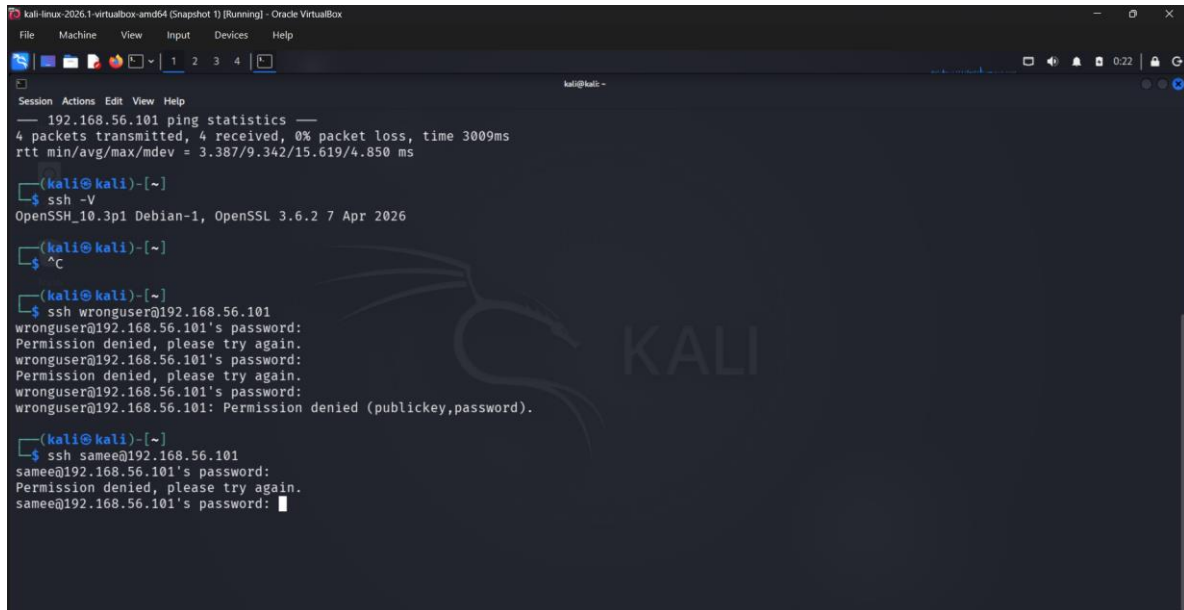


Figure 7: Ubuntu: `sshd -T` shows the effective SSH settings, then `auth.log` is read

Setting	Value seen	Meaning
passwordauthentication	yes	Users can log in with a password. This is what a brute-force attack targets.
permitrootlogin	prohibit-password	Root cannot log in with a password (only with a key). Good default.
peremptypasswords	no	Accounts with blank passwords are refused.

Step 3: Attack simulation (failed login attempts)

From Kali I attempted SSH logins with wrong passwords. I tried two cases: a user that does not exist (wronguser) and a real user with a wrong password (samee). These generate two different kinds of log entries, which is useful for detection because both patterns appear in real attacks.



```

kali@kali:~$ ping 192.168.56.101
--- 192.168.56.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 3.387/9.342/15.619/4.850 ms

kali@kali:~$ ssh -V
OpenSSH_10.3p1 Debian-1, OpenSSL 3.6.2 7 Apr 2026

kali@kali:~$ ssh wronguser@192.168.56.101
wronguser@192.168.56.101's password:
Permission denied, please try again.
wronguser@192.168.56.101's password:
Permission denied, please try again.
wronguser@192.168.56.101's password:
wronguser@192.168.56.101: Permission denied (publickey,password).

kali@kali:~$ ssh samee@192.168.56.101
samee@192.168.56.101's password:
Permission denied, please try again.
samee@192.168.56.101's password:

```

Figure 8: Kali: repeated "Permission denied" for wronguser and samee on 192.168.56.101

HOW TO READ THIS SCREENSHOT

Permission denied, please try again appears after each wrong password. After 3 wrong tries the client stops with Permission denied (publickey,password): the server offered two login methods, key and password, and both failed.

wronguser = username-guessing case (invalid user).

samee = valid username with a wrong password.

ETHICS NOTE

These attempts were run only against my own Ubuntu VM in an isolated lab. Doing this on any system without written permission is illegal.

Step 4: Log collection on the Ubuntu server

Every SSH login attempt is written by the system to /var/log/auth.log. I read the last lines with `sudo tail -5 /var/log/auth.log` and filtered for the word "failed" using `grep`.

```

samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$
samee@secops-server:~$ sudo sshd -T | grep -E 'Passwordauthentication|pubKeyauthentication'
samee@secops-server:~$ sudo sshd -T | grep password
permitrootlogin prohibit:password
passwordauthentication yes
permitemptypasswords no
samee@secops-server:~$ sudo tail -5 /var/log/auth.log
2026-09-28T02:07:57.589886+05:30 secops-server sshd-session[11136]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.102 user=samee
2026-09-28T02:07:59.992334+05:30 secops-server sshd-session[11136]: Failed password for samee from 192.168.56.102 port 42750 ssh2
2026-09-28T02:09:53.223849+05:30 secops-server sshd[10819]: Timeout before authentication for connection from 192.168.56.102 to 192.168.56.101, pid = 11136
2026-09-28T02:10:19.151196+05:30 secops-server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by samee(uid=1000)
2026-09-28T02:10:19.158631+05:30 secops-server sudo: samee : TTY=/dev/pts/0 ; PWD=/home/samee ; USER=root ; COMMAND=/usr/bin/tail -5 /var/log/auth.log
samee@secops-server:~$ sudo tail -5 /var/log/auth.log | grep "failed"
samee@secops-server:~$

```

Figure 9: Ubuntu auth.log: authentication failure and Failed password for samee from 192.168.56.102

Anatomy of one log line (from the screenshot):

Part	Example from screenshot	What it tells us
Timestamp	2026-09-28T02:07:59	When the attempt happened
Host	secops-server	Which machine recorded it
Process	sshd-session[11136]	The SSH server process and its process ID
Event	Failed password for samee	What happened (a failed login) and for which user
Source IP	from 192.168.56.102	WHO attacked (the key detail for blocking)
Port / protocol	port 42750 ssh2	Source port of the attacker and the SSH version used
PAM line	authentication failure; rhost=192.168.56.102 user=samee	PAM (Pluggable Authentication Modules) also records the failure
Timeout line	Timeout before authentication for connection from 192.168.56.102	The attacker connected but did not finish logging in

Step 5: Verifying the logs reached Wazuh

On the Wazuh server (IP 192.168.56.104 on eth0) I searched the alert files. The same failed-password events from Ubuntu were present, proving that the Wazuh agent forwards the logs and the manager analyses them.

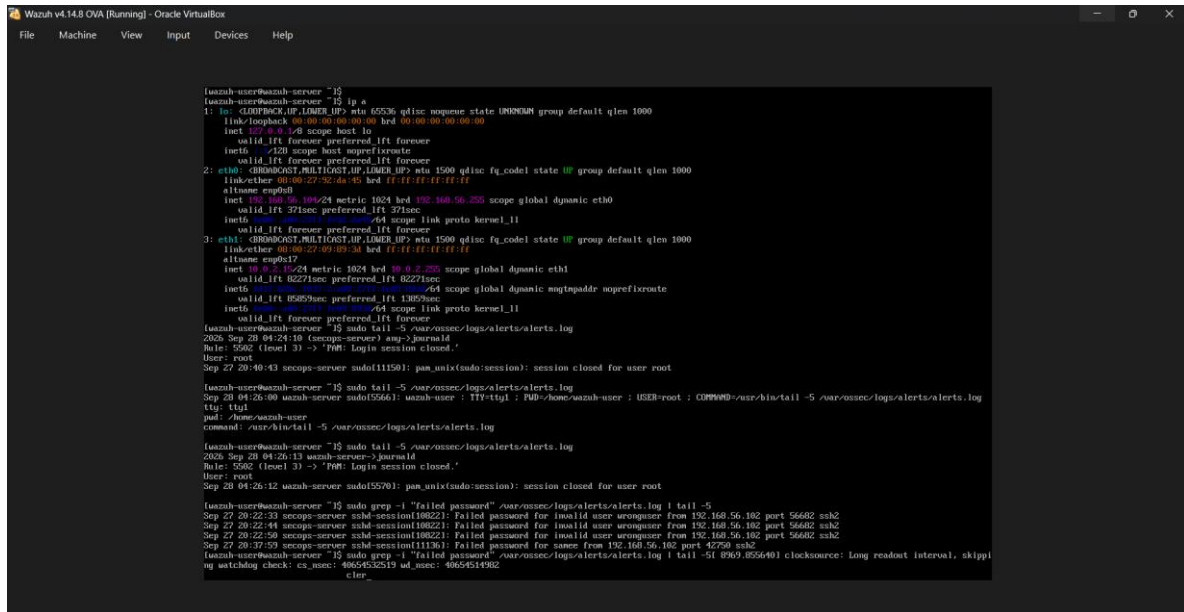


Figure 10: Wazuh server: `ip a` and `grep` of "failed password" in `alerts.log`

KEY EVIDENCE IN THIS SCREENSHOT

Three lines: **Failed password for invalid user wronguser from 192.168.56.102** (20:22:33, 20:22:44, 20:22:50), then **Failed password for samee** at 20:37:59.

The source IP is the same in every line: **192.168.56.102 (the Kali attacker).**

The log came from **secops-server** but was read on **wazuh-server**, so log forwarding works.

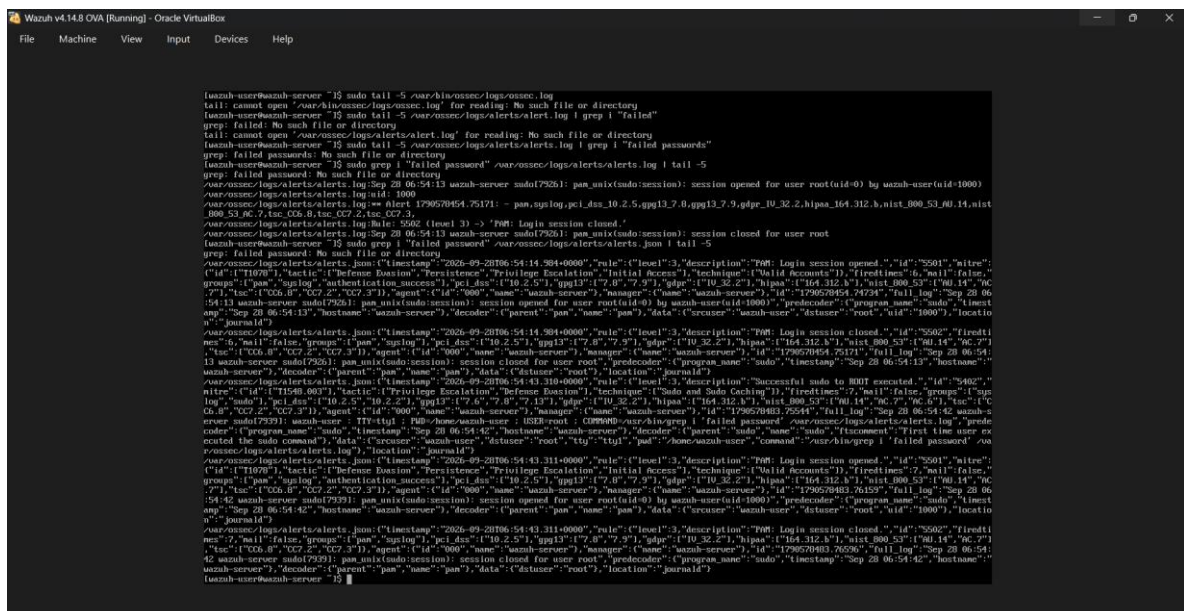


Figure 11: Wazuh alerts.json: structured alerts with rule id, level, MITRE technique and full log

The JSON alert format is more detailed than alerts.log. Each alert has a rule.id, rule.level, the MITRE ATT&CK tactic and technique, the agent name and the original full_log. For example rule 5501 "PAM: Login session opened", rule 5502 "Login session closed" and rule 5402 "Successful sudo to ROOT executed" (MITRE T1548.003).

Step 6: Wazuh dashboard analysis

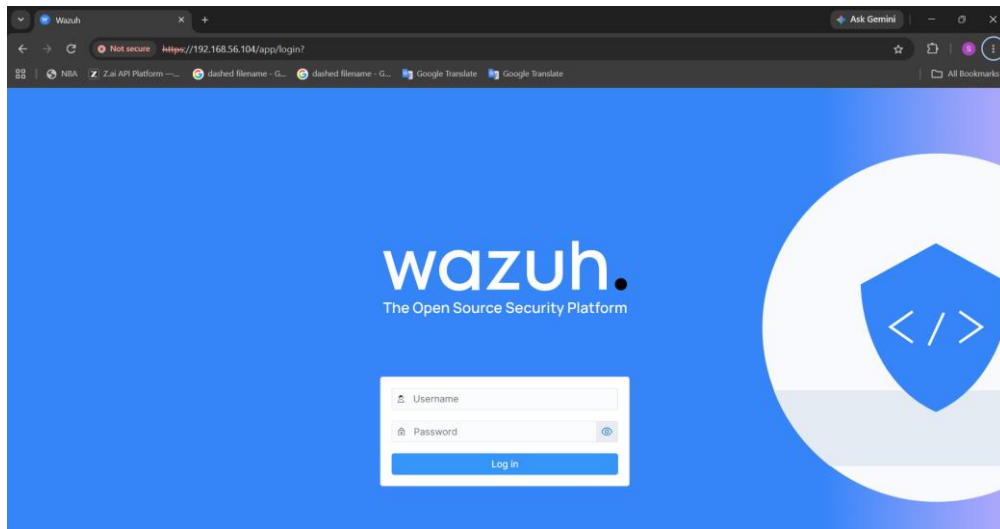


Figure 12: Wazuh web dashboard login at <https://192.168.56.104> (self-signed certificate, so the browser says "Not secure")

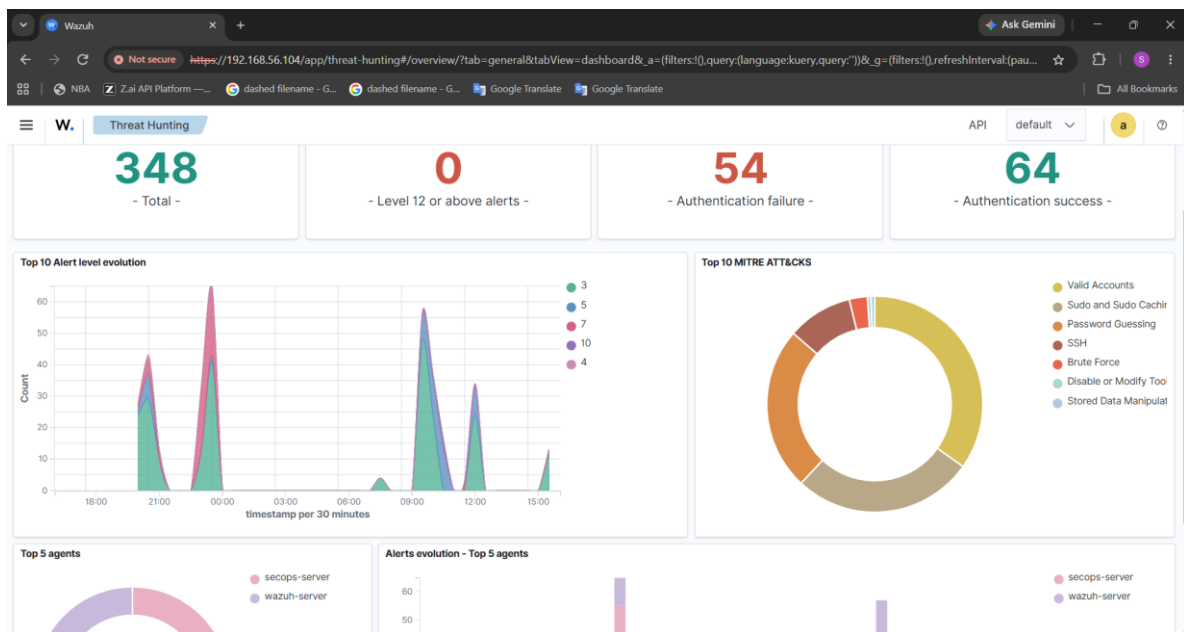


Figure 13: Threat Hunting overview: 348 alerts, 54 authentication failures, MITRE ATT&CK donut chart

Dashboard item	Value seen	Meaning
Total alerts (24 h)	348	All events that matched a Wazuh rule
Level 12 or above	0	No critical alerts; nothing severe happened
Authentication failure	54	Failed logins, including my attack attempts
Authentication success	64	Normal successful logins and sudo sessions

Dashboard item	Value seen	Meaning
Top MITRE ATT&CK	Valid Accounts, Sudo, Password Guessing, SSH, Brute Force	Attack techniques the alerts map to
Top 5 agents	secops-server, wazuh-server	Which machines produced the alerts

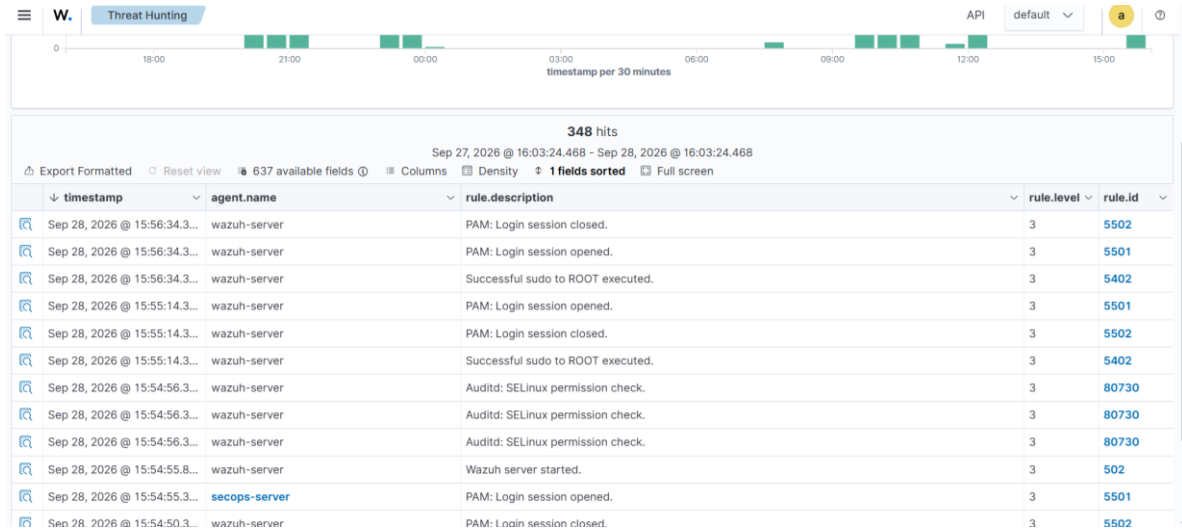


Figure 14: Threat Hunting Events tab: 348 hits with rule description, level and rule ID

4. Key Features

Feature	How it works	Problem it solves
Centralised log collection	Wazuh agent on Ubuntu sends auth.log events to the Wazuh server	Admin no longer needs to read logs on each machine by hand
Real-time alerting	Decoders and rules turn log lines into levelled alerts (level 3 info, 5 warning, 10 high)	Attacks are noticed in seconds, not days
MITRE ATT&CK mapping	Every alert is tagged with a tactic and technique (e.g. Brute Force, Password Guessing)	Uses a global standard language to describe the attack
Dashboard and threat hunting	Charts, counters and searchable events	Easy investigation of who, when and how many
Source-IP identification	Every failed login records the attacker IP and port	Gives the exact information needed to block the attacker
Response and blocking	Firewall rule or Wazuh Active Response bans the IP	Stops the attack instead of only watching it
Incident documentation	Structured report with timeline and evidence	Provides proof, accountability and learning

5. Incident Response Phase: Observe, Block and Document

5.1 Observing the events

After the attempts, I watched the alerts in the Wazuh dashboard (Threat Hunting > Events) and looked for the following signs of brute force:

- Many "Failed password" or "authentication failure" events in a short time.
- The same source IP repeated in all events (here 192.168.56.102).
- Attempts against invalid users (wronguser) and against real users (samee).
- A jump in the alert-level graph and in the MITRE techniques Brute Force and Password Guessing.
- Any success from the same IP after many failures, which would mean the attack worked.

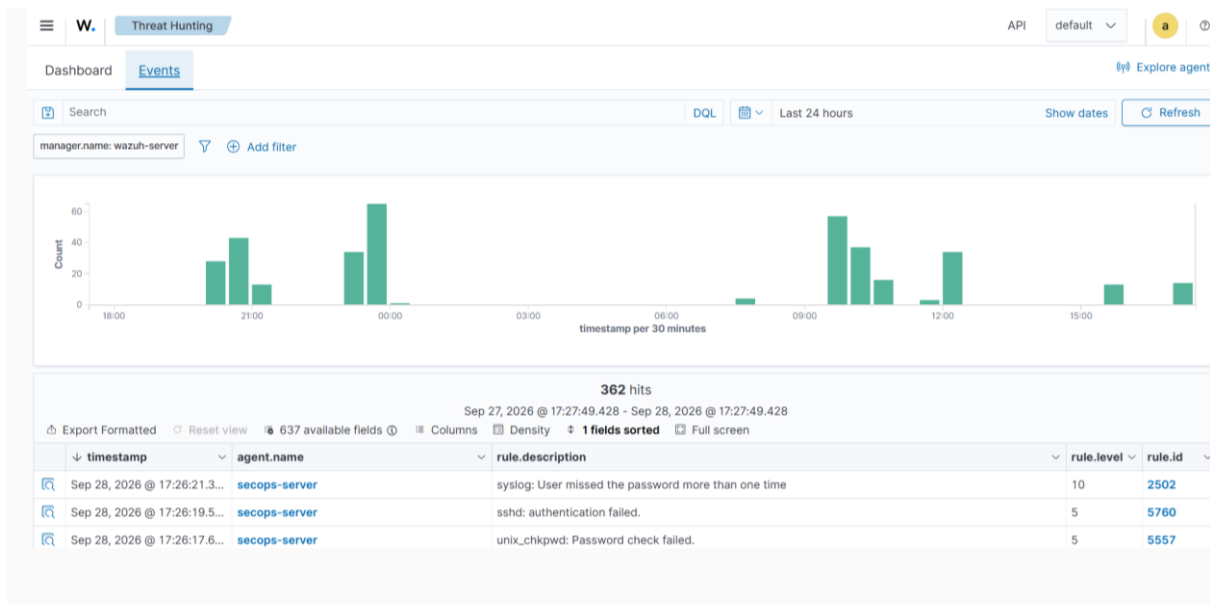


Figure 15: Observing the events in Wazuh after the attack

The screenshot shows the Wazuh Threat Hunting Events dashboard with a detailed view of a single event. The table lists 15 rows of events. The event details are as follows:

timestamp	agent.name	rule.description	rule.level	rule.id
Sep 28, 2026 @ 15:56:34.3...	wazuh-server	PAM: Login session closed.	3	5502
Sep 28, 2026 @ 15:56:34.3...	wazuh-server	PAM: Login session opened.	3	5501
Sep 28, 2026 @ 15:56:34.3...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Sep 28, 2026 @ 15:55:14.3...	wazuh-server	PAM: Login session opened.	3	5501
Sep 28, 2026 @ 15:54:14.3...	wazuh-server	PAM: Login session closed.	3	5502
Sep 28, 2026 @ 15:55:14.3...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Sep 28, 2026 @ 15:54:56.3...	wazuh-server	Auditd: SELinux permission check.	3	80730
Sep 28, 2026 @ 15:54:56.3...	wazuh-server	Auditd: SELinux permission check.	3	80730
Sep 28, 2026 @ 15:54:56.3...	wazuh-server	Auditd: SELinux permission check.	3	80730
Sep 28, 2026 @ 15:54:55.8...	wazuh-server	Wazuh server started.	3	502
Sep 28, 2026 @ 15:54:55.3...	secops-server	PAM: Login session opened.	3	5501
Sep 28, 2026 @ 15:54:50.3...	wazuh-server	PAM: Login session closed.	3	5502
Sep 28, 2026 @ 15:54:48.0...	wazuh-server	Listened ports status (netstat) changed (new port opened or closed).	7	533
Sep 28, 2026 @ 12:25:33.3...	wazuh-server	PAM: Login session opened.	3	5501
Sep 28, 2026 @ 12:25:33.3...	wazuh-server	PAM: Login session closed.	3	5502

Figure 16: Wazuh alert details for the brute-force attempt (expand one event)

5.2 Blocking criteria (when do we block an IP?)

We should not block on a single wrong password, because real users mistype passwords. A clear rule (criteria) avoids blocking innocent people and still stops real attackers. The following criteria was used in this project:

Criteria	Condition	Action
1. Repeated failures	5 or more failed logins from the same IP within 2 minutes	Mark as suspicious and raise a high-severity alert
2. Invalid-user guessing	Attempts using usernames that do not exist (e.g. wronguser)	Treat as reconnaissance; block IP if repeated
3. Success after failures	Successful login from an IP that just failed many times	Treat as compromise: block, lock the account, investigate
4. Known-safe IPs	Admin IP or trusted management host	Never auto-block; whitelist and only alert
5. Block duration	First offence 10 minutes (auto-unblock); repeat offender permanent block	Reduces damage from false positives

How the IP is blocked. Option A: manual firewall rule on the Ubuntu server (simple and good for a demo):

```
sudo ufw deny from 192.168.56.102 to any port 22
sudo ufw status numbered      # verify the rule exists
```

Option B: Wazuh Active Response (automatic). Wazuh has a built-in firewall-drop command. Adding a block like this to the manager ossec.conf makes it drop the IP on the agent when the brute-force rule fires, and unblock after the timeout:

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712</rules_id>
  <timeout>600</timeout>
</active-response>
```

IMPORTANT TERMS

Active Response = Wazuh automatically runs a script (like blocking an IP) when a rule triggers.

timeout 600 = the block lasts 600 seconds (10 minutes), then it is removed automatically.

False positive = a harmless event wrongly treated as an attack (for example an admin who forgot a password). Whitelisting and timeouts reduce this.

```

reset
status
status numbered
status verbose
show ARG
version

reset firewall
show firewall status
show firewall status as numbered list of RULES
show verbose firewall status
show firewall report
display version information

Application profile commands:
app list
app info PROFILE
app update PROFILE
app default ARG

list application profiles
show information on PROFILE
update PROFILE
set default application policy

samee@secops-server:~$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
22 DENY 192.168.56.102
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)

samee@secops-server:~$

```

Figure 17: Blocking the attacker IP (192.168.56.102)

```

kali@kali:~$ sudo systemctl status ssh
[sudo] password for kali:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-28 10:44:32 EDT; 8min ago
   Invocation: 4ae8ab504337497aad7bfd138a91687c
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 721 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 745 (sshd)
   Tasks: 1 (limit: 2112)
   Memory: 1.9M (peak: 2.9M)
   CPU: 34ms
   CGroup: /system.slice/ssh.service
           └─745 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 28 10:44:31 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Sep 28 10:44:32 kali sshd[745]: Server listening on 0.0.0.0 port 22.
Sep 28 10:44:32 kali sshd[745]: Server listening on :: port 22.
Sep 28 10:44:32 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

kali@kali:~$ ssh samee@192.168.52.101
ssh: connect to host 192.168.52.101 port 22: Connection refused

kali@kali:~$

```

Figure 18: Proof that the block works

5.3 Documentation criteria (what must be written after the incident?)

A security incident is not finished until it is documented. Good documentation proves what happened, helps the next analyst, and is required for audits and lessons learned. The documentation must be Accurate, Complete, Timely, Clear and backed by Evidence.

Criteria	What to record	Why it matters
Who	Attacker IP 192.168.56.102, targeted users (wronguser, samee), analyst name	Identifies the source and the responsible person
What	SSH brute-force / password-guessing attempts	Defines the type of incident
When	Start and end time from logs (e.g. Sep 27 20:22 onward)	Builds the timeline

Criteria	What to record	Why it matters
Where	Target secops-server 192.168.56.101, detected by wazuh-server 192.168.56.104	Shows which assets were affected
How detected	Wazuh agent forwarded auth.log; rules 5710 / 5716 / 5503 and MITRE Brute Force	Explains the detection method
Evidence	Screenshots, log lines, alert IDs, dashboard counts	Proof that can be verified later
Impact	No successful attacker login, no level 12+ alerts, no data loss	Measures the damage
Action taken	IP blocked (command, time, duration)	Shows containment was done
Root cause + fix	Password authentication enabled; move to keys, fail2ban, rate limits	Prevents the incident from repeating

Severity classification

Severity	Wazuh level	Example	Response
Low	3 to 4	Session opened / closed, single sudo	Log only
Medium	5 to 7	Single failed password, invalid user attempt	Monitor
High	10+	Multiple failures from one IP (brute force)	Block IP + document
Critical	12+	Successful login after brute force / root compromise	Isolate host, full investigation

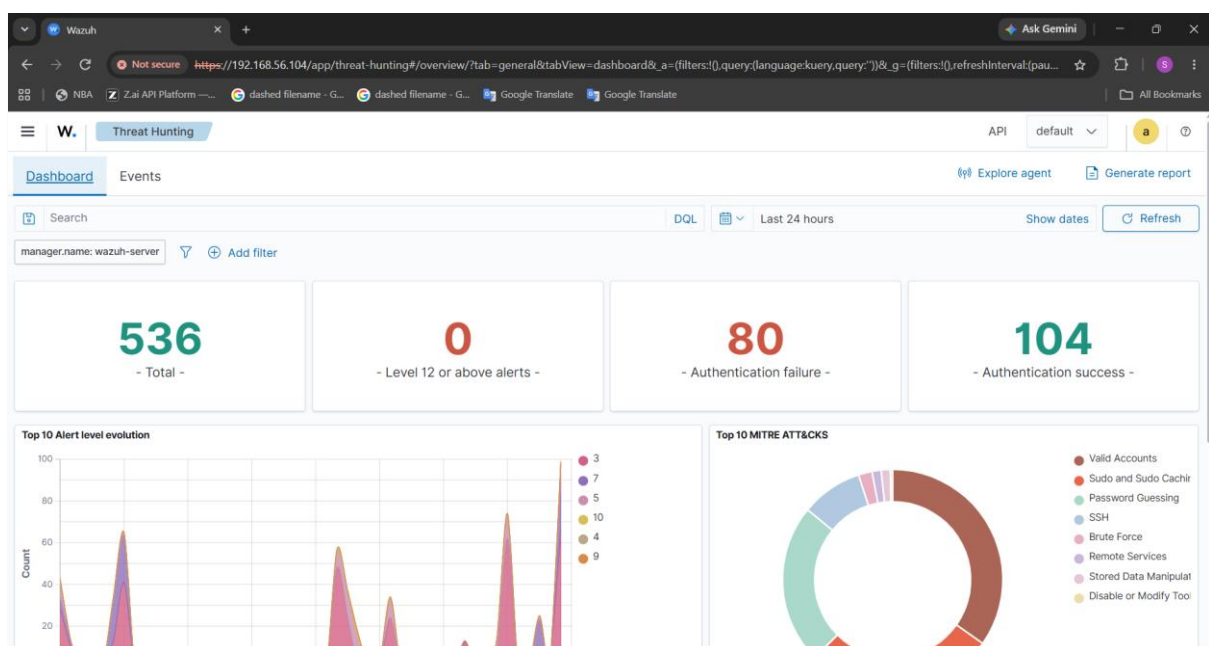


Figure 19: Documentation after the incident: Wazuh alerts/summary for the final report

SSH Brute-Force Incident Report

Field	Details
Incident	SSH Brute-Force Attack
Target	Ubuntu Server (192.168.56.101)
Attacker IP	192.168.56.102
Service	SSH
Port	22
Detection Tool	Wazuh
Response Tool	UFW Firewall

Incident Timeline

Step	Event	Result
1	Multiple failed SSH login attempts were generated from 192.168.56.102.	Wazuh detected the suspicious SSH activity.
2	The source IP was identified as 192.168.56.102.	Attacker IP identified.
3	UFW rule was created to block the IP on SSH port 22.	22 DENY IN FROM 192.168.56.102
4	The UFW rule was verified.	Firewall status showed the deny rule as active.
5	A new SSH connection was attempted from Kali to Ubuntu.	Connection was refused.
6	The block was confirmed.	SSH access from 192.168.56.102 was successfully prevented.

FINAL RESULT

The suspicious IP address 192.168.56.102 was successfully blocked from accessing the Ubuntu server through SSH port 22 using UFW. A new SSH connection attempt from Kali resulted in **Connection refused**, confirming that the firewall response was effective.

Evidence

- Wazuh SSH alert
- UFW deny rule
- UFW status showing the active rule

- Kali SSH attempt showing **Connection refused**

Figure 20: Completed SSH brute-force incident report

6. Technologies Used

Technology	Used for	Why it was chosen
VirtualBox	Running 3 virtual machines on one laptop	Free, supports host-only networks and snapshots for safe testing
Kali Linux	Attacker machine and SSH client	Standard penetration-testing distribution
Ubuntu Server	Victim server with the SSH service	Very common real-world server OS
OpenSSH	Remote login service (port 22)	Most widely used SSH implementation, and the usual brute-force target
Wazuh v4.14.8	SIEM: log collection, rules, alerts, dashboard	Free, open source, has MITRE mapping and active response
Linux commands	ping, ssh, sshd -T, tail, grep, ip a	Fast way to verify network, configuration and logs
auth.log / alerts.json	Raw evidence and structured alerts	Original logs plus enriched SIEM data
UFW / iptables	Blocking the attacker IP	Built-in Linux firewall, simple to use
MITRE ATT&CK	Naming attack techniques	Global standard used by security teams

7. Challenges Faced

Challenge	How it was solved
Wrong log file names: alerts.log versus alerts.json (tail and grep returned "No such file")	Listed /var/ossec/logs/alerts/ and used the correct file names; alerts.json holds the detailed JSON alerts
grep for "failed password" found nothing in some files because of case and wording	Used grep -i (ignore case) and searched with smaller, exact words
Making three VMs reach each other	Used a host-only network (192.168.56.0/24) and confirmed each IP with ip a and ping
Too many level 3 events (PAM sessions, sudo) hiding the real attack events	Filtered by rule description, source IP and level
Kernel message "clocksource: Long readout interval" on the Wazuh console	Recognised it as a harmless VirtualBox timing message, not a Wazuh fault
Limited laptop RAM while running Kali, Ubuntu and Wazuh together	Used VM snapshots and started only the machines needed for each step

8. Achievements

- Built a working three-VM SOC (Security Operations Center) lab.
- Confirmed two-way connectivity with 0% packet loss.
- Generated real failed-login events and traced them from Kali, to Ubuntu auth.log, to Wazuh alerts.
- Identified the attacker IP, targeted usernames and MITRE techniques from the evidence.
- Defined clear blocking and documentation criteria and completed the incident report.
- Finished the complete detect, observe, block and document cycle.

9. Future Plans

- Enable Wazuh Active Response so blocking is fully automatic.
- Add fail2ban and SSH key-only login with password authentication disabled.
- Use larger, automated attack simulations and tune the Wazuh rule thresholds.
- Add email or Slack alerts for level 10 and above.
- Extend to other attacks: web logins, port scans, file integrity monitoring and vulnerability detection.
- Connect this project with PacketVision, my PCAP analysis project, so network captures support the log evidence.

10. Personal Contribution

I did this project individually, so I carried out every stage myself:

- Designed and built the lab: Kali, Ubuntu server and Wazuh OVA on VirtualBox.
- Tested the network with ping in all directions and checked the SSH configuration with sshd - T.

- Performed the SSH login attempts and collected the logs on Ubuntu and Wazuh.
- Analysed the alerts in the terminal (alerts.log, alerts.json) and in the Wazuh dashboard.
- Prepared the flowchart, blocking criteria, documentation criteria and the final incident report.

11. Lessons Learned

- **Technical:** how SSH authentication is logged, how a SIEM decodes logs, how rule levels map to severity, and how MITRE ATT&CK describes attacker behaviour.
- **Technical:** attack and defence are two sides of the same log line: the same "Failed password" entry is the attacker's attempt and the defender's evidence.
- **Practical:** always verify each step (network, service, logs) before moving on; small errors like a wrong file name waste time.
- **Personal:** patience and documentation are as important as tools; a screenshot taken at the moment is worth more than memory later.
- **Security mindset:** a strong password policy and key-based SSH prevent most brute-force attacks before monitoring is even needed.

12. Conclusion

This project shows that a brute-force attack can be simulated, detected, blocked and documented using free open-source tools. Wazuh turned scattered log lines into clear, mapped alerts, the source IP 192.168.56.102 was identified, and clear blocking and documentation criteria made the response consistent and evidence-based. The key message is that detection alone is not enough: defenders must also respond and record. This lab is a strong base for a real SOC analyst workflow.

Appendix A: Abbreviations

Abbreviation	Full form	Simple meaning
SIEM	Security Information and Event Management	Central system that collects and analyses security logs
SSH	Secure Shell	Encrypted remote login (port 22)
SOC	Security Operations Center	Team/room that monitors security alerts
IP	Internet Protocol	Address of a machine on a network
ICMP	Internet Control Message Protocol	Protocol used by ping
TTL	Time To Live	How many hops a packet may travel; hints at the OS
PAM	Pluggable Authentication Modules	Linux login framework that writes auth events
OVA	Open Virtual Appliance	Ready-made virtual machine file
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge	Catalogue of attacker methods
UFW	Uncomplicated Firewall	Easy front-end for Linux firewall rules
MFA	Multi-Factor Authentication	Login needs more than a password
CIDR	Classless Inter-Domain Routing	Notation like /24 for the network size

Thank You

Detecting and Responding to SSH Brute-Force Attacks using Wazuh

Any questions?

Sameeha shajahan | Student at Brototype